



Full Launch Audit

The deployed site · the complete codebase · project documents

The 72-hour plan · marketing plan and unit economics

14 September 2026 · every figure verified directly

Youssef Alsherief

botyalla.com

info@youssefalsherief.tech

Contents

00 Thirty-second summary

01 The live site — what I checked myself

02 The code — what must be fixed before the campaign

03 The documents — what they say and what they hide

04 The 72-hour plan — exactly what to do

05 Measurement — the item without which you spend blind

06 The marketing plan — economics before channels

07 Risks — honestly

08 The direct answer to your question

Date: 13 September 2026 · **Site:** <https://botyalla.com> — **live and serving** **Scope:** the deployed site · the complete codebase (25 test files · 190KB in `app.py` alone) · 11 documents in `docs/` · a marketing plan

How I read this project: every number here I verified myself — I ran the full test suite, checked the live headers with `curl`, and *proved* the money bugs by executing the code rather than reading it. I did not rely on memory from earlier sessions: the project doubled in size in two days (`app.py` went from 102KB to 190KB), so I re-pulled everything from your machine before forming any judgement.

Thirty-second summary

The good news: the product is stronger than you think. 24 of 25 test files pass (**the one failure is an image I didn't copy**, not a defect in your code). The security is genuinely production-grade: I probed 16 routes with an attacker account against another user's bot — **all returned 404**. Ownership is enforced inside the query, money operations are atomic, migrations are safe, and the WhatsApp webhook signature fails closed.

The news that matters more: do not run a marketing launch the day after tomorrow as things stand. Not because the product is weak — but because three things would turn a successful campaign into damage:

1

The homepage promises something that is switched off

The hero and all marketing copy are built around one-tap bot creation, and the feature is **disabled** until you press a button in BotFather (a two-minute step). Visitors arrive from your ad and meet a broken promise.

2

You will spend and learn nothing
There is no analytics of any kind on the site. No Google Analytics, no alternative. You will not know which channel brought signups, or where they dropped off.

3

Three defects detonate under the first traffic spike Detailed in §2 — all of them are hours of work, not weeks.

Verdict: the product is 90% ready. Give yourself **72 hours instead of 48**, work the list in §4, and launch on **17 September** with confidence instead of on the 15th with anxiety. The difference is two days — and it is the

difference between a launch you build on and a launch you redo.

The live site — what I checked myself

✓ What is genuinely excellent

I inspected the live headers, and the result is better than most Egyptian platforms:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-...'
Set-Cookie: session=...; Secure; HttpOnly; Path=/; SameSite=Lax
```

- **HTTPS works and HTTP 301-redirects** to it.
- **COOKIE_SECURE=1 is actually set** — the cookie carries the **Secure** flag. That closes the L-02 item that was still open.
- **CSP with a nonce and no **unsafe-inline** on scripts** — rare to see implemented this carefully.
- **SEO is built with real care:** well-written Arabic **title** and **description** · **canonical** · a complete **og:** set with a 1200×630 image · **hreflang** for all three values (ar/en/x-default) · valid **JSON-LD** with a correct **Organization** · working **robots.txt** and **sitemap.xml** .
- **Compression is on** (gzip) for HTML and static files: the homepage transfers 20KB instead of 95KB.

● Three marketing mistakes in the site itself

1. **robots.txt** hides your pricing page from Google.

```
Disallow: /pricing
```

Pricing is **the highest purchase-intent page in any SaaS**. Someone searching "WhatsApp bot price Egypt" is your ready-to-pay customer, and you are stopping Google from finding you. It is missing from **sitemap.xml** too. Delete the line, add the page to the sitemap — immediate return at zero cost.

2. **www** and the apex domain both return 200 with no redirect.

https://botyalla.com/ and **https://www.botyalla.com/** both serve as separate copies. Google sees duplicate content and splits your link equity between them, and sessions don't carry across (a user signs in on one and finds themselves signed out on the other). Pick one and 301 the other.

3. Static files are revalidated on every single visit.

```
/static/dist/chunk-src.js    326KB    Cache-Control: public, max-age=0, must-revalidate
```

326KB + 202 + 91 are revalidated with the browser on **every** page load. The cause is fixed filenames (`console.js` , not `console.a3f9.js`), which can't be cached safely. Fix: content hashes in the filenames (one line in `vite.config.js`) then `max-age=31536000, immutable` . Repeat-visit load time roughly halves — and that matters on an Egyptian mobile connection.

Minor note: `og-ar.png` is **337KB**. WhatsApp and Facebook accept it, but compressing to ~150KB makes the link preview appear faster when shared — and that preview is the first thing anyone receiving your link sees.

The code — what must be fixed before the campaign

I ran the full suite against the latest version from your machine:

✓ 24 of 25 files pass · ~470 tests

Sole failure: static/brand/og-ar.png — an image I didn't copy (present on the server, verified)

That is an excellent number. But tests cover what you thought to test; what follows I found outside them.

● Critical — fix before any campaign

C-1 · A campaign that reaches customers and is refunded in full

In `bot_manager.py:318` the broadcast is bounded by a timeout, and on expiry it returns `(0, len(peers))` — "nothing was delivered" — so `app.py:1015` refunds the whole amount. **But the messages are actually sent**, because `asyncio.run_coroutine_threadsafe(...).result(timeout=)` **does not cancel the coroutine** — it only stops waiting for it.

Proven by executing the code:

```
TimeoutError → caller reports (0, n) = all failed, full refund  
but the coroutine actually delivered: [0, 1, 2, 3, 4]
```

The time budget is `len(peers) × 0.6` seconds, while the per-message WhatsApp timeout is **15 seconds**. A 50-subscriber campaign gets 30 seconds — **two slow responses from Meta are enough**. Result: the customer gets their campaign free and you pay Meta. And it is deliberately repeatable.

C-2 · The receipt routes have no rate limit — and they are the most expensive thing you run

`/subscribe/<plan>` and `/wallet/topup` run synchronous OCR with a **30-second** timeout inside the request. You run `workers=1, threads=4`.

Four users uploading a receipt at the same moment = **all four threads busy**, and the fifth sees a frozen site. Worse: the asyncio loop running **every one of your customers' bots** lives in the same process — so it stops too.

Note that `ai_setup`, `ticket` and `asset_url` are all rate-limited. These two alone are not — an oversight, not a choice.

C-3 · Blocking network calls with no limit

`/api/validate-token`, `/bot/create` and `/account/link-telegram` wait on Telegram/Meta for 10–15 seconds with no rate limit. Four concurrent requests make the whole site unresponsive. Additionally `/api/validate-token` accepts any token and returns Telegram's verdict — a free oracle for bulk-checking stolen bot tokens.

C-4 · An AI route with no quota burns your API bill

`app.py:1250` — `ai_setup` has neither `_rate_limited` nor a quota check, while its sibling `ai_session_start` has both. Each call is up to two Gemini/Groq requests **on the platform's key**. Registration is open, so any registered user can loop it and burn your credit.

C-5 · Charging a customer's wallet for a reply that never arrived

`flow_engine.py:253` — the AI reply price is deducted **before** sending, and the result of `ch.send_text` is **never checked**. The WhatsApp channel returns `None` on every failure: the 24-hour window closed · Meta returned an error · plan limit hit · network failure.

The code **knows** this — `LoggedChannel._log` checks that exact case and skips logging the outbound message. But the piastres were taken and are not returned. A silent, repeating loss, and the first thing a WhatsApp customer will complain about.

C-6 · Broadcast outruns the nginx timeout, so the owner pays twice

`nginx.conf` sets `proxy_read_timeout 60s`, and the broadcast is synchronous, sending one message every 0.05s. **~110 Telegram subscribers** exceeds sixty seconds. nginx cuts the connection, the owner sees an error while **the credit is already deducted and sending is still in progress** — so they press "send" again. Double charge, duplicate campaign to their customers.

Important — fix in the first week

#	Issue	Impact
I-1 1	Missing indexes on every hot table	I ran <code>EXPLAIN QUERY PLAN</code> : <code>events</code> , <code>leads</code> , <code>orders</code> , <code>payments</code> and <code>bots</code> are all full scans . With <code>workers=1</code> , every slowdown hits everyone.
I-2 2	The <code>events</code> table is never purged	Written on every <code>/start</code> across all platform bots, and the maintenance cycle cleans everything except it. <code>stats_daily</code> pulls every event since inception, then filters 14 days in Python.
I-3 3	Formula injection in CSV export	A customer name like <code>=HYPERLINK(...)</code> is written raw. The victim is your paying customer when they open the export in Excel.
I-4 4	Export silently truncated at 300 rows	A shop owner exports "all orders" and gets the last 300 with no warning.
I-5 5	500 on ordinary input	A non-numeric <code>working_days</code> raises <code>ValueError</code> — every other numeric field in that function is guarded except this one.
I-6 6	<code>/logout</code> on GET	<code></code> on any site signs your visitor out. A nuisance rather than a breach, but it violates an invariant in your own guide.

✓ What I checked and found sound — don't spend time here

Item	Result
Ownership	16 routes probed with an attacker account — all 404 . Enforced inside the query, not after it.
Roles	Every admin route is guarded, and owner-only features really are hidden from <code>support</code> .
Money	<code>finalize_payment</code> is one atomic connection · <code>_wallet_move</code> locks with <code>WHERE balance >= ?</code> · <code>ai_reply_allow</code> reserves within the same UPDATE. No findings.
Migrations	All idempotent and guarded. Fernet key rotation never overwrites a row it failed to decrypt — careful work.
Secrets in code	None. Scanned for every known key pattern. <code>.env</code> really is outside git.
SQL injection	None. Every value is a bound parameter.
SSRF	<code>asset_store</code> protection is solid: DNS pinned to the connection and every redirect re-checked.
<code>/healthz</code>	Actually checks the database and returns 503 on failure.

The documents — what they say and what they hide

I read 11 documents. The findings that matter:

● `WEBSITE_LAUNCH.md` has 28 items and not one is ticked

And they are not cosmetic. The most serious:

- **Egyptian Personal Data Protection Law 151/2020:** the document itself says you "will likely need a **licence**" from the Data Protection Centre and a designated data protection officer, that review "can take up to **90 working days**", and that the compliance deadline is **1 November 2026**. That cannot be done in two days — but it also does not block launching today. **Start the process now**, because the duration is outside your control.
- **SPF + DKIM + DMARC are not configured.** Without them your receipts and password-reset emails land in Spam. A user who cannot recover their account is a user you have lost.
- `PUBLIC_URL` : without it **no reset link is sent at all** (a deliberate fail-closed). Verify the server's copy.

● **The feature your marketing rests on is switched off**

`ONE_TAP_SETUP.md` is explicit:

"The homepage talks about one-tap creation — must I enable it before launch? **Yes**. The hero, the steps and the FAQ are now built around one-tap creation, so enable **Bot Management Mode** before you publish the site to people."

The code is complete and tested. What is missing is **three manual minutes**: BotFather → your platform bot → Bot Settings → enable Bot Management Mode → then "Save" in platform settings. Confirm the line turns ● enabled.

⚠ And note a documented bug: on **macOS** the link opens the wrong chat (bugs.telegram.org/c/60634). The only workaround is opening from mobile via the QR code. **Test it yourself on two devices before the campaign.**

● Stale documents that contradict reality

- `USER_GUIDE.md` still describes only the manual path: "message @BotFather → `/newbot` → copy the token". Your user guide **doesn't know about** the feature you are marketing.
- `ARCHITECTURE.md` **never mentions WhatsApp**, nor `channels/`, nor the inbox.
- `USER_GUIDE.md`, `HOSTINGER.md` and `README.md` say the admin password is `admin / admin1234`, while `PRODUCTION.md` says it is randomly generated. **Rotate `ADMIN_PASS` now — the old one is in git history.**
- A contradiction that directly affects marketing: `README.md` says the free plan gets an "offline generator", while `TESTER_FEEDBACK_PLAN.md` says it gets **the real agent with a 3-session quota**. The second is far stronger commercially — unify the message.

● A documented functional gap, unresolved

From `TESTER_FEEDBACK_PLAN.md` under "deliberately deferred": the canned replies sent by the **store / booking / FAQ** templates on Telegram **do not appear in the inbox**. The business owner sees their customer's message but not their own bot's reply to it. Not a launch blocker, but it will generate support tickets — know it before a customer tells you.

The 72-hour plan — exactly what to do

Day 1 (14 Sep) — money and safety fixes · 6–8 hours

#	Task	Time	Why now
1	Fix C-1 (broadcast refund on timeout)	1.5h	Every campaign from now on risks a direct loss
2	Rate-limit <code>/subscribe</code> , <code>/wallet/topup</code> , <code>/api/validate-token</code> , <code>/bot/create</code> (5–10/hour)	1h	Prevents the whole platform being taken down
3	Lower the tesseract timeout from 30s to 10s	5m	A phone receipt reads in under 3
4	Quota + limit on <code>ai_setup</code> (two lines) or delete the route	20m	Protects your API bill
5	Refund piastres when an AI reply fails to send	45m	A silent, repeating loss
6	Move broadcast to a background thread + one-campaign-per-bot lock	2h	Prevents the double charge
7	Add the eight indexes (one migration)	30m	Every hot query is a full scan today

Day 2 (15 Sep) — operations and measurement · 5–6 hours

#	Task	Time
8	Enable Bot Management Mode and test on mobile and desktop	30m
9	Install analytics — see §5	1h
10	Remove <code>Disallow: /pricing</code> and add it to the sitemap	10m
11	301 between www and apex	20m
12	Content hashes in filenames + long cache	45m
13	Rotate <code>ADMIN_PASS</code> · verify <code>PUBLIC_URL</code>	20m
14	SPF + DKIM + DMARC · test on mail-tester.com (target 9/10)	1.5h
15	UptimeRobot on <code>/healthz</code> every 5 minutes	15m
16	One restore drill from a backup	45m

Day 3 (16 Sep) — human verification · 4 hours

#	Task
17	The full journey yourself on a real phone: register → one-tap create → set welcome → start → chat with the bot → request a subscription → approve as admin → receipt arrives by email
18	Repeat with three people who don't know the product and watch in silence. Every question they ask is copy that needs rewriting
19	ar/en pages · the four policies · the 404 page
20	PageSpeed Insights on the homepage (mobile)
21	Update <code>USER_GUIDE.md</code> for one-tap creation, and unify the free-plan message

Start today in parallel (outside your control on time)

- **Commercial register and tax card** — fees are EGP 40.5 for individuals; about two weeks (the tax file is the long pole).
- **Meta Tech Provider** — 3 to 6 weeks of waiting.

- **Data protection licence** — up to 90 working days.

None of these block your Telegram launch. They block only the **commercial WhatsApp channel**.

Measurement — the item without which you spend blind

There is no analytics on the site today. This is the single most dangerous marketing gap in this report, because it does not merely spoil the campaign — it stops you learning **why** it went wrong.

What I specifically recommend

Use [Plausible](#) or [Umami](#), not Google Analytics. The reasons are practical, not ideological:

1. **Your CSP does not permit third-party scripts** without loosening it. Umami can be self-hosted on your own server, keeping `script-src 'self'` clean.
2. **Egypt's Law 151/2020** — cookieless analytics dramatically simplifies your obligation, and you must comply before November anyway.
3. A 1KB script instead of 45KB — that matters on an Egyptian mobile connection.

The events to instrument from day one

Without these five, your campaign is guesswork:

Event	The question it answers
<code>signup_started</code> → <code>signup_completed</code>	How many are lost in the signup form itself?
<code>bot_create_started</code> → <code>bot_live</code>	Your single most important number. What share reach a working bot?
<code>pricing_viewed</code> → <code>subscribe_started</code> → <code>payment_uploaded</code>	Where does a paying user stop?
<code>first_customer_message</code>	When does the bot reach its first real conversation? That is the "I get it" moment
Signup source (UTM)	Which channel brings people who pay, not just people who register

The governing ratio: `signup_completed` → `bot_live` . Below 40%, your problem is the product, not marketing, and spending more multiplies the loss. Above 60%, spending is justified.

The marketing plan — economics before channels

6.1 The number that governs everything

These are not imported market figures — they are computed directly from your own pricing:

Plan	Monthly	Annual (~30%)	Marginal margin
Free	0	0	zero cost — Telegram has no per-message fee
Merchant	299	2,510	~100% (Telegram only)
WhatsApp	899	7,550	18%–100% depending on message mix
Agency	2,999	25,190	High

The central truth of your model: the free plan on Telegram costs you **zero marginally**. No per-message fee; the only cost is server memory (I measured it: **3.44 MB per bot** using your own [tools/loadtest_bots.py](#)).

That means the **free tier is not an acquisition cost** — it is the **acquisition channel itself**. Every working free bot is a salesperson for you in front of its owner's customers.

6.2 The numbers you need in order to decide on spend

Metric	Value	Derivation
Maximum paid acquisition cost	≤ EGP 300	Merchant 299 × ~6 months average retention = 1,794. The 3:1 LTV:CAC rule gives a 598 ceiling. Start conservative at 300.
Break-even per customer	One month	EGP 299 covers a 300 acquisition cost in month one
Target conversion (free → paid)	4%–8%	The common SaaS band for a self-serve product
Sustainability point	~35 Merchant subscribers	≈ EGP 10,465/month covers server, domain, email and a safety margin

Practical conclusion: your first goal is not a thousand users. It is **35 paying subscribers**. At 5% conversion that means **700 signups** — achievable without a large ad budget.

6.3 Channel order — by return, not by noise

🏆 Phase 1 (weeks 1–2): ten design partners, by name

Do not launch publicly first. Pick **ten merchants you know personally** or through contacts, from clear categories: a clothing shop · a small restaurant · a salon · a clinic · an Instagram store.

Why this before advertising: - You will find ten usability problems before a thousand people see them. - You will get **real stories with names and numbers** — which is the entire raw material of your later marketing. - It costs nothing.

The offer: the paid plan free for 3 months in exchange for real usage · a 20-minute call every two weeks · permission to use their name and results.

Success criterion: 7 of 10 reach a working bot, and 5 are still using it after two weeks.

🏆 Phase 2 (weeks 3–4): the communities where your merchants already are

Your Egyptian merchant lives in specific places:

Channel	Why it works	How
Facebook merchant groups ("shop owners", "online sellers Egypt", shipping and payments groups)	A free concentration of high intent	Don't advertise. Answer real questions for two weeks first, then mention the product only where it is an actual answer
TikTok and Reels — a 30-second video: "a bot for your shop in one minute"	A "show me and I'll believe you" audience, and one-tap creation is excellent visual material	Record your literal screen: the tap → the bot works → a real conversation. No loud music, no effects
The bot itself	Every working free bot is seen by its owner's customers	A "by BotYalla" footer on the free plan (already built — the merchant removes it by upgrading)
Affiliates (already in the platform)	Small marketing agencies already have the customers	Commission on the first 3 months, with a transparent dashboard

🏆 Phase 3 (month two onward): paid search — after you know your numbers

Do not buy ads before measurement works. Then start with search, not display:

- Someone typing "I want a bot for my shop" has intent. Someone scrolling past a Facebook ad does not.

- Start at **EGP 1,500 per week** with long, specific keywords: "order bot for restaurants", "WhatsApp auto-reply for stores".
- **Kill any keyword above EGP 300 per signup**, however promising it looks.

6.4 The message — what to say precisely

A governing rule: never promise what you haven't built. An earlier review established that the bot **does not collect money from the merchant's customer** — there is no payment field in the `orders` table. So do not say "we collect payments from your customers for you." What you do have is stronger and true:

✓ Say	✗ Don't say
"Your bot is live in a minute — no app to install, no line of code"	"An integrated automation platform" (meaningless)
"Pay with Vodafone Cash or InstaPay — no international card"	"Innovative business solutions"
"In Arabic, and it understands your customer's dialect"	Any promise of collecting payments from the merchant's customer
"Free forever on Telegram"	"The best in the Middle East" with no evidence

Your real differentiation is three things, all of them built: local payment (every competitor demands an international card) · Arabic natively rather than translated · one-tap creation.

6.5 Content plan — 30 days

1 "How it works" 3 short videos: one-tap creation · setting the welcome · the first conversation	2 Use cases One post per category: restaurant · clothing · clinic · salon. Each solves one problem	3 Design partners The first two real stories with names and numbers (with permission)	4 Objections "Is it expensive?" · "Do I need a developer?" · "Is my data safe?"
--	--	--	---

Tone rule: write the way you would speak to a merchant in his shop — not the way you'd write a brochure. "Your bot is live in a minute" beats "a simplified setup experience."

6.6 First-month budget

Item	Amount
Search ads (after measurement)	EGP 6,000
Filming and editing (simple, on your phone)	0
Design-partner incentives	0 (free subscriptions)
Tools (analytics · monitoring · SMTP)	~EGP 500
WhatsApp message credit reserve	EGP 3,000
Total	~EGP 9,500

Target: 700 signups → 35 subscribers → costs covered from month one.

Risks — honestly

Risk	Likelihood	Impact	What to do
Launching before one-tap is enabled	High if you launch the day after tomorrow	Every visitor meets a broken promise	3 minutes in BotFather + test on two devices
Spending without measurement	Certain without \$5	You lose the money and the knowledge	Install analytics first
A campaign delivered and refunded (C-1)	Medium, rising with growth	A direct, repeatable loss	1.5-hour fix
Receipt congestion freezes the platform (C-2)	High on launch day	Full outage on the money path	Rate limit + shorter timeout
The macOS one-tap bug	Certain on macOS	Mac users fail	Put "best from mobile" on the page + QR
Law 151/2020	Certain by November	Legal exposure	Start the process this week
Solo founder	Ongoing	You are the single point of failure	Document the restore, prepare canned support replies

The direct answer to your question

"Can I publish the site to the market the day after tomorrow?"

Technically: the site is already published and running with good security. The real question is whether you should point a marketing campaign at it the day after tomorrow.

My answer: no — give it 72 hours. The difference is not in the quality of the product, but in three things that turn a successful campaign into damage:

1. **The homepage's promise is switched off** (3 minutes to fix)
2. **No measurement** — you would spend and learn nothing (1 hour to fix)
3. **Three money/availability defects** that detonate under the first spike (one working day)

That is **a day and a half of actual work**. After it, you launch onto a product that is measured and protected, instead of fixing things under pressure while customers watch.

And one piece of advice that isn't about code: "dominating the market" is not a measurable goal. Your measurable goal for the first ninety days is **35 paying subscribers and two real, named success stories**. Whoever reaches those two reaches the next thousand with a single campaign. Whoever jumps straight at a thousand with no measurement and no stories burns the budget and learns nothing.

The product you have built deserves a good launch. Give it two more days.

Appendix: what I verified directly for this report

Claim	How I verified it
Site is live and secure	<code>curl</code> against the live headers
24/25 test files pass	Full <code>run_tests.sh</code> run on the latest copy from your machine
C-1 (broadcast delivered and refunded)	Executed the code and demonstrated the behaviour , not read it
Indexes missing	<code>EXPLAIN QUERY PLAN</code> on the hot queries
Ownership enforced	16 routes with an attacker account — all 404
500 on <code>working_days</code>	An actual call producing <code>ValueError</code>
<code>robots.txt</code> blocks <code>/pricing</code>	Reading the live file
No analytics	Analysing the deployed page's HTML
One-tap is disabled	The literal text of <code>ONE_TAP_SETUP.md</code>

What I did not verify: the state of `.env` on the server · whether you enabled Bot Management Mode after that document was written · Egyptian market-size figures (I found no reliable primary source in the time available, so I avoided inventing numbers — every figure in \$6 is computed from your own pricing).